מכון טכנולוגי חולון
Holon Institute of Technology

COMPUTER SCIENCE DEPARTMENT מחלקה למדעי המחשב

סדנה מתקדמת בתכנות 61108
סמסטר ב' תשפ"ו

מטלה 1

מערכים, מבנים, רשימות מקושרות

שאלה 1:

כתבו פונקציה בשם squareSums המקבלת שני מספרים טבעיים n1 ו- n2 (n2>n1) אשר מוצאת את כל המספרים הנמצאים בטווח של n1 עד n2(כולל n1 ו-n2) שסכום הספרות של כל אחד מהם הוא מספר ריבועי מושלם.
מספר ריבועי מושלם הוא מספר שהשורש שלו הוא מספר שלם.

נגדיר את המבנה הבא לייצוג המספר:

```
typedef struct number
{
        unsigned long long num;
        int sum;
} Number;
```

בו num הוא המספר עצמו ו- sum הוא סכום ספרותיו.

לאחסון התוצאות על הפונקציה לבנות מערך דינאמי של איברים מטיפוס Number כך שעבור שדה num של כל איבר המערך, $n1 \le num \le n2$ ו- num עונה לתנאי הנ"ל.
הפונקציה תחזיר את כתובת המערך ותעביר (by reference) את גודלו.
במידה והמערך לא מכיל אף איבר, כתובתו תהיה NULL וגודלו יהיה 0.

דוגמא:

עבור n1=31, n2=20, הפונקציה תיצור את המערך הבא:

Num	22	27	31
Sum	4	9	4

ותעביר 3.

הבדיקה הכפולה עבור כל מספר האם סכום ספרותיו הוא מספר ריבועי היא לא יעילה.
מצד שני הקצאה זמנית של המערך בגודל המקסימלי n2-n1+1 היא לא חסכונית. לכן יש **להכפיל פי 2 את גודל הזיכרון המוקצה** לפי הצורך תוך כדי **מילוי המערך** (ראו את השיטה במצגת 1).

ניתן להניח שיש בזיכרון מספיק מקום להקצאה.
חתימת הפונקציה היא:
Number *squareSums(unsigned long long n1, unsigned long long n2, int *p_size);

יש להשתמש בפונקציות עזר הבאות:

א. פונקציה int isPerfectSquare (int num) אשר מקבלת כפרמטר מספר טבעי num ובודקת האם הוא ריבועי מושלם. אם כן, הפונקציה תחזיר 1. אחרת עליה להחזיר 0.
ב. פונקציה int digitSum(unsigned long long num) אשר מחזירה את סכום הספרות של num.

שאלה 2:

נגדיר את **שכנים של איבר** A[i][j] של מטריצה A כאיברים A[i +1][j], A[i -1][j], A[i][j + 1], A[i][j - 1]. אם A[i][j] נמצא בשורה\עמודה ראשונה\אחרונה של A אז מספר שכניו יהיה פחות בהתאם.

כתבו פונקציה בשם matrixSumNeighbor שמקבלת מטריצה A של מספרים שלמים המיושמת כמערך דו-ממדי סטטי וגדליה.
על הפונקציה להקצות מערך דו-ממדי חדש המיישם מטריצה B בעלת אותם גדלים כמו A כך שכל איבר B[i][j] שלה יהיה שווה **לסכום של שכניו** A[i][j].

דוגמא:

$$A: \begin{bmatrix} 1 & 3 & 2 \\ 3 & 2 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

$$B: \begin{bmatrix} 6 & 5 & 3 \\ 4 & 8 & 5 \\ 7 & 4 & 2 \end{bmatrix}$$

הפונקציה תחזיר את הכתובת של המערך החדש.

ניתן להניח שיש בזיכרון מספיק מקום להקצאה.

חתימת הפונקציה היא:

int ** matrixSumNeighbor(int A[][COLS], int rows, int cols);
כך ש- COLS הוא קבוע המוגדר באמצעות #define.

יש להשתמש בפונקציות עזר הבאות:

א. פונקציה int neighborSum(int A[][COLS], int rows, int cols, int i, int j) אשר מקבלת מערך דו-ממדי סטטי עם גדליו וגם קואורדינטות i ו-j של איברו ומחזירה את סכום שכניו של A[i][j].
ב. פונקציה int **allocMatrix (int rows, int cols) אשר מקבלת את גדלי המערך הדו-ממדי, מקצה אותו ומחזירה את כתובתו.

שאלה 3:

נגדיר מטריצה דלילה עם ערכי קיצון:
מטריצה דלילה עם ערכי קיצון היא מטריצה דלילה שבה, אם בסדר הסריקה הסטנדרטי (שורה־שורה) מתקיים שלכל איבר $a_{ij} \neq 0$,ערכו הוא קיצון (*min/max*) ביחס לקבוצת האיברים הלא־אפסיים שנצפו עד לאותו שלב.

דוגמא:

המטריצה הבאה הינה מטריצה דלילה עם ערכי קיצון:

i \ j	0	1	2	3	4
0	0	4	5	0	0
1	0	0	2	0	7
2	0	0	0	0	1
3	0	0	0	9	0

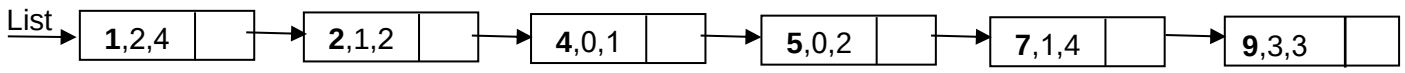
בסדר סריקה רציף:
4 ראשון, 5 מקסימאלי, 2 מינימאלי, 7 מקסימאלי, 1 מינימאלי, 9 מקסימאלי.

כתבו פונקציה בשם createSortedSparseList המקבלת מטריצה דלילה עם ערכי קיצון A כמערך דו־ממדי דינאמי ואת ממדיה, ובונה רשימה מקושרת **ממוינת לפי ערכיה** המכילה את כל האיברים השונים מאפס.

כל איבר ברשימה מכיל שלושה נתונים. כל שלשה תכיל את ערכו של איבר המטריצה והקואורדינטות i ו-j (מס' השורה ומס' העמודה) של אותו האיבר במטריצה.

הפונקציה תעביר (reference by) את הרשימה הממוינת (ללא איברי דמה בייצוג סופי), ותחזיר את גודלה (כמות האיברים ששונים מ-0)

עבור הדוגמא מעלה, הפונקציה תיצור את הרשימה הממוינת הבאה (סדר האיברים ברשימה חשוב!!):



הפונקציה תחזיר את הערך 6 (כמות האיברים ששונים מ-0)

ניתן להניח שיש בזיכרון מספיק מקום להקצאה. **יש לבצע מעבר אחד על המטריצה.**

למימוש השלשה יש להשתמש במבנה הבא:

```
typedef struct triad
{
        int value ,i, j;
} Triad;
```

ובהתאם לממש את איבר הרשימה באופן הבא:

```
typedef struct item
{
	Triad data;
	struct item *next;
} Item;
```

חתימת הפונקציה היא:

```
int createSortedSparseList (int **A, int rows, int cols, Item **lst);
```

יש להשתמש בפונקציות עזר הבאות:

א. פונקציה Triad createThree(int i, int j, int value) המקבלת שלושה מספרים שלמים ומחזירה שלשה אחת המורכבת משלושת הפרמטרים שקיבלה.

ב. פונקציה בשם insert להקצאה והוספת איבר חדש לרשימה. יש לקבוע את חתימתה על פי רצונכם.

<u>הוראות:</u>

1. ההגשה בזוגות בלבד! סטודנט שאין לו בן-זוג להגשה ולא מוצא כזה יש לפנות למרצה לעזרה בנושא.

2. **רק** אחד משני בני-הזוג מעלה את הפתרון לתיבת ההגשה. בן הזוג השני מעלה קובץ טקסט(Notepad) עם הפרטים המלאים של שני בני הזוג (שם + ת.ז.). לדוגמא:

Avraham Avrahami 0987654321
Michael Levy 1234567890

3. יש להשתמש כתבנית **רק** בקובץ Assignment_1_template.c המצורף אשר מכיל חלק מהקוד. צריך לשתול בתוכו את הטקסטים\קוד של כל הפונקציות הנדרשות במקום המתאים. **אין** להגיש קובץ או תיקייה מכווצת, עבודה שתוגש כך לא תיבדק.

4. בהתאם לתבנית יש לאחד את כל השאלות הנ"ל לתוכנית אחת באמצעות תפריט הפונקציות Ex1(), Ex2(), Ex3() המשמשות להפעלת פונקציות השאלות 1,2,3 בהתאמה ומטפלות בהכנה ובקליטה של הנתונים המקוריים, בהדפסת התוצאות ובשחרור זיכרון דינאמי.

5. בפונקציה Ex1() יש להצהיר על משתנים, לקלוט את הפרמטרים המקוריים, לקרוא לפונקציה squareSums ולהציג את התוצאות. להדפסת המערך להשתמש בפונקציית עזר:

```
void printArray(Number *arr, int size);
```

בסוף יש לשחרר את הזיכרון המוקצה.

6. בפונקציה Ex2() יש להצהיר על משתנים (כולל המערך המקורי), לבצע אתחול ראשוני עם ערכים קבועים למערך הדו-מימדי באמצעות פונקציית עזר:

```
void inputMatrix(int A[ ][COLS], int rows, int cols);
```

ולהדפיס את המערך הנקלט באמצעות פונקציית עזר:

```
void printMatrix(int A[][COLS],int rows,int cols);
```

לאחר מכן לקרוא לפונקציה matrixSumNeighbor
ולהציג את המערך הדו-ממדי החדש באמצעות פונקציית עזר:

```
void printDynamicMatrix(int **A, int rows, int cols);
```

בסוף יש לשחרר את הזיכרון באמצעות פונקציית עזר:

```
void freeMatrix (int **A, int rows);
```

7. בפונקציה Ex3() יש להצהיר על משתנים, לקלוט את גדלי המערך הדינאמי המקורי ולהקצות אותו (ניתן להניח שיש בזיכרון מספיק מקום להקצאה) באמצעות פונקציית עזר allocMatrix (שבה משתמשים גם בשאלה 2). לאחר מכן לקלוט את המערך באמצעות פונקציית עזר:

```
void inputDynamicMatrix(int **A, int rows, int cols);
```

ולהדפיס את המערך הנקלט באמצעות פונקציית עזר:

```
void printDynamicMatrix(int **A, int rows, int cols);
```

בשלב הבא צריך לקרוא לפונקציה createSortedSparseList ולהציג את הרשימה הבנויה באמצעות פונקציית עזר:

```
void printList(Item *lst);
```

בסוף יש לשחרר את הזיכרון באמצעות פונקציות עזר:

```
void freeMatrix (int **A, int rows);
void freeList (Item* lst);
```

8. אפשר להשתמש בפונקציות עזר נוספות לפי שיקול דעתכם.

9. אין צורך בבדיקת תקינות הקלט ולא צריך לבדוק בתוך פונקציה את תקינות הפרמטרים שלה.

10. יש להקפיד על ממשק ידידותי ככל האפשר.

11. יש להשתמש בשמות משמעותיים ולתעד את הקוד עם הערות(הערות באנגלית בלבד). יש להקפיד לכתוב בצורה מבנית.

12. תכנית שלא עוברת קומפילציה לא תתקבל!

בהצלחה!